# § 2. Дерево отрезков и проектирование агрегата

Пусть массив меняется, а запросы требуют суммы произвольных диапазонов. Прямой просмотр делает обновления дешёвыми, а запросы дорогими. Префиксные суммы меняют роли: запрос становится постоянным, зато изменение одного элемента затрагивает длинный хвост. Дерево отрезков хранит информацию на нескольких масштабах и даёт логарифмическую стоимость обеих операций.

После главы вы сможете построить дерево, доказать оценку запроса, реализовать точечные изменения и спроектировать сведения узла для максимальной суммы подотрезка. Предполагается понимание рекурсии и оценок первой главы.

## 2.1. Динамическая задача и статический ответ

Контракт: `set(p,x)` присваивает элементу $a_p$ значение $x$; `sum(l,r)` возвращает $\sum_{i=l}^{r-1} a_i$. Индексы удовлетворяют $0 \le p < n$ и $0 \le l \le r \le n$. На пустом диапазоне сумма равна нулю. Размер массива не меняется. Добавление элемента в конец — другой интерфейс, которого текущая реализация не обещает.

Для статического массива определим $P_0 = 0$, $P_{i+1} = P_i + a_i$. Тогда ответ равен $P_r - P_l$. Однако присваивание в позиции $p$ меняет все $P_j$ с $j > p$. Если обновления часты, предвычисления приходится слишком часто чинить.

| Представление | Построение | Присваивание | Сумма диапазона |
|---|---|---|---|
| Только массив | $O(n)$ | $O(1)$ | $O(r-l)$ |
| Префиксные суммы | $O(n)$ | $O(n)$ | $O(1)$ |
| Дерево отрезков | $O(n)$ | $O(\log n)$ | $O(\log n)$ |

Таблица не назначает победителя без знания нагрузки. При одном запросе после миллиона присваиваний обычный массив может быть выгоднее. Структуру выбирают по полному набору операций и памяти, а не по наличию слова «логарифм».

**Ход из видео.** В записи 13.09.2022 первые девять минут посвящены интерфейсу и конфликту префиксных сумм с обновлениями. Далее массив разбивается на интервалы разных размеров. Именно эту последовательность сохраняем. Числовой массив ниже — редакторский пример, а не восстановленные значения с доски.

## 2.2. Интервалы становятся узлами

Пусть $a = [2, 7, 1, 4, 3, 6, 5, 8]$. Сохраним суммы соседних пар, затем четвёрок и всего массива. Получится двоичное дерево. Лист отвечает за одну позицию; внутренний узел — за объединение двух соседних интервалов. Корень хранит 36, его дети — 14 и 22.


(c) Aleksey Tolstikov

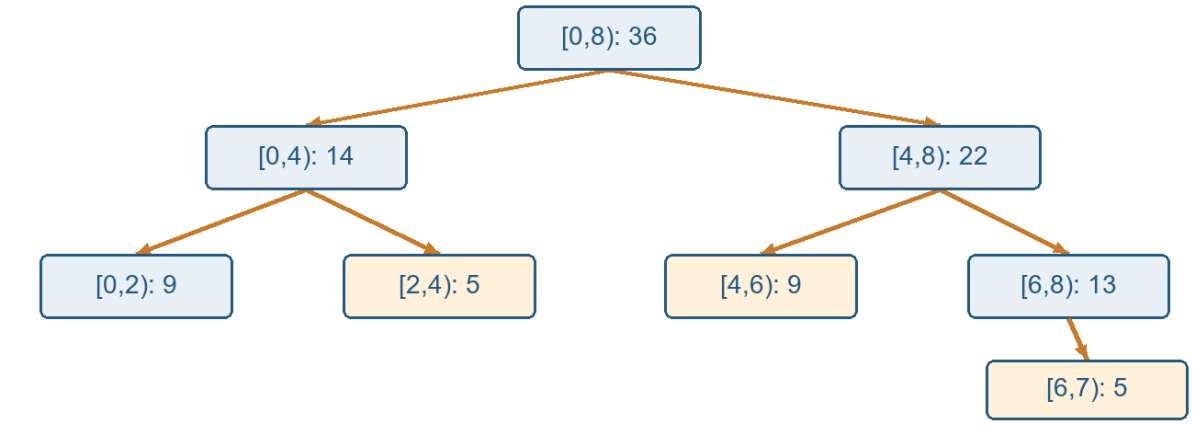



Рисунок 1: Узел хранит сумму своего диапазона; запрос собирается из непересекающихся узлов.

Инвариант: значение узла равно сумме элементов его фиксированного интервала. Оно не описывает диапазон **значений** элементов: индексы остаются прежними, даже если содержимое массива сильно меняется.

Для запроса $[2, 7)$ берём интервалы $[2, 4)$, $[4, 6)$ и $[6, 7)$. Их суммы 5, 9 и 5 дают 19. Элементы не теряются и не учитываются дважды. Запрос $[0, 8)$ совпадает с корнем и не требует спуска, а $[3, 4)$ в конце спуска совпадает с одним листом.

Если размер не является степенью двойки, есть два удобных подхода. Рекурсивное дерево делит фактический диапазон пополам. Итеративное дополняет массив до ближайшей степени двойки нейтральными элементами. В этой главе выберем второй вариант: он делает хранение и код компактными.

Пусть `base` — минимальная степень двойки не меньше $n$. В массиве `t` корень имеет индекс 1, дети узла $v$ — $2v$ и $2v + 1$, листья начинаются с `base`. Для $n > 0$ выполняется $n \le base < 2n$, значит размер `2*base` линейный. При $n = 0$ оставляем `base=1`; единственный допустимый запрос — пустой.

## 2.3. Почему запрос логарифмический

Рекурсивное объяснение выделяет три случая. Если интервал узла не пересекается с запросом, возвращаем ноль. Если целиком лежит внутри, возвращаем сохранённую сумму. При частичном пересечении спускаемся в детей и складываем их ответы.

Нельзя обосновывать $O(\log n)$ только высотой: дерево высоты $\log n$ содержит $\Theta(n)$ узлов. Важен характер посещённых вершин. На каждом уровне не более двух узлов пересекают границы запроса частично. Остальные посещённые узлы либо целиком принимаются, либо сразу отбрасываются. От каждого граничного пути отделяется не более одного целиком принимаемого поддерева на уровне. Поэтому число существенных посещений ограничено константой на уровень.

Итеративный алгоритм выражает ту же идею через два указателя на границы полуинтервала. Если левая граница — правый ребёнок, её узел нельзя объединить с левым братом: тот уже вне запроса. Забираем узел и двигаем границу вправо. Если правая граница нечётна, узел непосредственно слева от неё целиком входит в запрос, а его правый брат уже не входит. Забираем этот узел. После этого обе границы поднимаются к родителям.

Для $[2, 7)$ при `base=8` начинаем с $(l, r) = (10, 15)$. Узел 14 соответствует $[6, 7)$, он попадает в правый накопитель. После подъёма границы равны $(5, 7)$. Узел 5 даёт $[2, 4)$, узел 6 — $[4, 6)$. Следующий подъём даёт равные границы и завершает запрос.

| Шаг | Границы в массиве дерева | Добавленные интервалы |
|---|---|---|
| Листья | 10, 15 | $[6, 7)$ |
| Уровень пар | 5, 7 | $[2, 4)$ и $[4, 6)$ |
| Завершение | 3, 3 | Ничего |

На каждом уровне выполняется постоянное число операций, а число подъёмов логарифмическое. Это доказательство относится к одному запросу в худшем случае, а не к амортизированной стоимости серии запросов.

## 2.4. Полная реализация суммы

```
class SumTree {
    int n, base = 1;
    std::vector<long long> t;
public:
    explicit SumTree(const std::vector<long long>& a) : n(int(a.size())) {
        while (base < n) base *= 2;
        t.assign(2 * base, 0);
        for (int i = 0; i < n; ++i) t[base + i] = a[i];
        for (int v = base - 1; v > 0; --v)
            t[v] = t[2 * v] + t[2 * v + 1];
    }
    void set(int p, long long x) {
        assert(0 <= p && p < n);
        p += base;
        t[p] = x;
        while (p > 1) {
            p /= 2;
            t[p] = t[2 * p] + t[2 * p + 1];
        }
    }
    long long sum(int l, int r) const {
        assert(0 <= l && l <= r && r <= n);
        long long left = 0, right = 0;
        l += base; r += base;
        while (l < r) {
            if (l & 1) left = left + t[l++];
            if (r & 1) right = t[--r] + right;
            l /= 2; r /= 2;
        }
        return left + right;
    }
};
```

Полный исходник с заголовками находится в `code/02-segment-tree.hpp`. Размер и индексы представлены `int`: учебный контракт предполагает $n \le 10^6$, что также исключает переполнение `base` и индексов. Значения и суммы используют `long long`; вызывающая сторона гарантирует отсутствие переполнения сумм. `assert` проверяет предусловия в отладочной сборке и не заменяет валидацию недоверенного ввода в приложении.

Построение заполняет листья, после чего обходит внутренние узлы от последнего к корню. Дети всегда уже готовы, поэтому инвариант устанавливается индукцией по высоте. Это $O(base) = O(n)$ работы, а не $O(n \log n)$: мы не делаем $n$ отдельных вызовов `set`.

Присваивание меняет один лист и всех его предков. Для нашего массива `set(3,6)` меняет сумму $[2, 4)$ с 5 на 7, сумму $[0, 4)$ с 14 на 16 и общую сумму с 36 на 38. Все остальные узлы остаются правильными: их диапазоны не содержат позицию 3.

Запрос не изменяет дерево, поэтому помечен `const`. Два накопителя выглядят избыточными для суммы, но сохраняют порядок частей. Это пригодится при обобщении на операцию, которая ассоциативна, но не коммутативна.

## 2.5. Сначала алгебра объединения

Дереву нужна операция `merge`, которая получает сведения двух соседних диапазонов и строит сведения их объединения. Для запросов с пустыми частями нужен нейтральный элемент. Ассоциативность означает, что разные способы расставить скобки не меняют результат, пока порядок частей сохранён.

Для суммы нейтральный элемент — 0; для минимума — достаточно большое значение или отдельный признак пустоты; для конкатенации строк — пустая строка. Коммутативность не обязательна. Если хранить конкатенацию, левый накопитель пополняют справа, а правый — слева. Перестановка частей дала бы другой ответ.

Не всякая операция подходит в исходном виде. Среднее двух средних не равно среднему объединения, если размеры частей различаются. Правильные сведения — пара «сумма, число элементов», а среднее вычисляется после объединения. Аналогично один минимум не позволяет узнать, сколько раз он встретился: необходимо хранить минимум вместе с кратностью.

Для пары $(m, c)$ при объединении выбираем меньший минимум и его счётчик; если минимумы равны, складываем счётчики. Нейтральная запись имеет счётчик 0 и специальный признак пустоты либо безопасно выбранное значение. Не используйте фиктивное бесконечное значение в арифметике, где его сложение может переполниться.

Дерево отрезков полезно тем, что позволяет отделить доказательство структуры от доказательства сводки. Сначала покажите, что `merge` точно описывает конкатенацию двух последовательностей. Затем алгоритм покрытия запроса автоматически объединит правильные сведения нужных частей.

## 2.6. Максимальная сумма подотрезка

Одна сумма недостаточна, чтобы найти лучший подотрезок. Например, последовательности $[5, -5]$ и $[0, 0]$ имеют одинаковую сумму 0, но разные максимальные суммы непустых подотрезков. При склейке нужен ответ на вопрос, выгодно ли продолжать подотрезок через границу.

Разрешим **пустой** подотрезок с суммой 0. Для каждого диапазона храним четыре числа: общую сумму `sum`, лучшую сумму префикса `pref`, суффикса `suff` и произвольного подотрезка `best`. Для одиночного значения $x$ три последних поля равны $\max(0, x)$.

Если сведения левой и правой частей обозначены $L$ и $R$, то

$$
\begin{aligned}
S &= L.sum + R.sum, \\
P &= \max(L.pref, L.sum + R.pref), \\
Q &= \max(R.suff, R.sum + L.suff), \\
B &= \max(L.best, R.best, L.suff + R.pref).
\end{aligned}
$$

Префикс либо заканчивается слева, либо включает всю левую часть и префикс правой. Суффикс разбирается симметрично. Лучший подотрезок расположен целиком в одной части либо пересекает границу и складывается из суффикса слева и префикса справа. Эти случаи исчерпывают все возможности.

```
struct Summary {
    long long sum, pref, suff, best;
};
Summary leaf(long long x) {
    long long p = std::max(0LL, x);
    return {x, p, p, p};
}
Summary merge(Summary l, Summary r) {
    return {l.sum + r.sum,
            std::max(l.pref, l.sum + r.pref),
            std::max(r.suff, r.sum + l.suff),
            std::max({l.best, r.best, l.suff + r.pref})};
}
```

Нейтральная запись — `{0,0,0,0}`. Для массива $[-2, 3, -1, 4, -5]$ ответ равен 6: подотрезок $[3, -1, 4]$. Если все элементы отрицательны, ответ 0 соответствует разрешённому пустому подотрезку. Для задачи с обязательным непустым ответом нужно изменить базу и обработку пустых узлов, а не только последнюю строку вывода.

**Связь с видео.** В лекции 08.09.2021 около 1:19:27 видны поля суммы и граничных оптимумов. Это источник идеи расширенной сводки. Обозначения, соглашение о пустом ответе и приведённые формулы здесь сформулированы и доказаны заново; они не являются цитатой неразборчивой доски.

## 2.7. Когда одного числа в узле мало

Рассмотрим запрос: сколько элементов на диапазоне меньше $x$? Сумма, минимум и максимум по отдельности не дают ответа. Один из вариантов для статического массива — хранить в каждом узле отсортированный список всех его элементов. Запрос раскладывается на $O(\log n)$ узлов, в каждом выполняется `lower_bound`.

Получаем $O(\log^2 n)$ времени запроса, $O(n \log n)$ памяти и столько же времени построения при слиянии дочерних списков. Каждый элемент присутствует один раз на каждом уровне. Название merge-sort tree отражает это сходство со слиянием, но дерево используется для запросов по исходным позициям.

Точечные изменения усложняют такой вариант. Если список узла — `std::vector`, вставка или удаление может сдвигать линейное число элементов. Поэтому нельзя механически объявить обновление логарифмическим только потому, что меняется логарифмическое число узлов. Стоимость локального действия в узле должна входить в итоговую оценку.

Количество **различных** значений также не является суммой количества различных в двух половинах: одинаковое значение может встретиться по обе стороны границы. Подход зависит от ограничений. Маленький универсум допускает битовые множества; статические запросы допускают специальные offline-методы; персистентные частотные деревья появятся позднее. Эта граница защищает от неверного универсального обещания «любая статистика за логарифм».

## 2.8. Отладка и задания

Начните с массива длины 0, 1, 3 и 8. Проверяйте пустой диапазон, весь массив, первый и последний элементы. Размер 3 обнаруживает ошибки в дополнении до степени двойки. После серии присваиваний сравнивайте все диапазоны с прямым суммированием на маленьком массиве.

Для сводки максимальной суммы независимо переберите все подотрезки, включая пустой. Кроме правильного ответа полезно проверять ассоциативность `merge` на коротких случайных массивах: объединения $(A + B) + C$ и $A + (B + C)$ должны совпасть по всем четырём полям. Такой тест проверяет смысл сводки, а не только конкретный маршрут запроса.

**Практика.** Реализуйте интерфейс суммы и затем замените сводку на «минимум и число его вхождений». Для углубления добавьте запрос максимальной суммы подотрезка. Это учебные задания данной главы, а не заявления о новых опубликованных задачах iRunner.

В следующей главе изменение затронет целый диапазон. Пересчитать все его листья было бы слишком дорого. Чтобы сохранить логарифмическое время, придётся хранить не только ответ, но и ещё не переданное детям преобразование.

## 2.9. Разбор: сводка и порядок частей

Проследим четыре поля максимальной суммы на массиве $[-2, 3, -1, 4, -5]$. Разделим его на $L = [-2, 3]$ и $R = [-1, 4, -5]$. Для левой части сумма 1, лучший префикс 1, лучший суффикс 3 и лучший подотрезок 3. Для правой части сумма −2, лучший префикс 3, лучший суффикс 0 и лучший подотрезок 4. Нулевой суффикс здесь пустой: любой непустой суффикс правой части имеет отрицательную сумму.

| Часть | sum | pref | suff | best |
|---|---|---|---|---|
| $[-2, 3]$ | 1 | 1 | 3 | 3 |
| $[-1, 4, -5]$ | −2 | 3 | 0 | 4 |
| Всё вместе | −1 | 4 | 1 | 6 |

Пересекающий границу кандидат равен $3 + 3 = 6$: суффикс $[3]$ слева продолжается префиксом $[-1, 4]$ справа. Лучший префикс всего массива равен $1 + 3 = 4$, а лучший суффикс — $-2 + 3 = 1$. Так четыре поля описывают разные оптимумы; ни одно нельзя без доказательства заменить другим.

Почему операция ассоциативна, хотя формулы содержат несколько максимумов? Сводка однозначно определяется последовательностью. Корректная `merge` возвращает сводку конкатенации. Поэтому `merge(merge(A,B),C)` и `merge(A,merge(B,C))` описывают одну последовательность и совпадают по всем полям. Это семантическое доказательство проще прямого раскрытия вложенных максимумов. Оно действует для корректных сводок последовательностей; произвольные четыре числа могут не удовлетворять их инвариантам.

Теперь рассмотрим строки как наглядную модель некоммутативной операции. Для запроса к буквам `abcdefgh` на $[1, 7)$ ответ должен быть `bcdefg`. При обходе снизу вверх правые части обнаруживаются от правого края к левому. Поэтому очередной кусок добавляется перед уже собранной правой строкой. Если добавлять его после, можно получить переставленные блоки, хотя набор выбранных листьев будет правильным.

Строковая модель годится для проверки порядка, но буквальная конкатенация не имеет постоянной стоимости. Если дерево хранит реальные строки, время слияний и объём скопированных символов надо учесть отдельно. Оценка $O(\log n)$ предполагает сводку постоянного размера и постоянную стоимость её объединения. Четыре числа этому условию удовлетворяют, строка переменной длины — нет.

## 2.10. Углубление: поиск по сохранённым суммам

Дерево может отвечать не только на запрос известного диапазона. Пусть все элементы неотрицательны, а их общая сумма положительна. Требуется найти первую позицию $p$, для которой сумма $[0, p + 1)$ не

меньше заданного $k$, где $1 \le k \le sum(0, n)$. Если элементы равны 0 или 1, это поиск позиции $k$-й единицы.

Начинаем в корне. Если сумма левого ребёнка не меньше $k$, ответ находится слева. Иначе весь левый диапазон можно пропустить: вычитаем его сумму из $k$ и переходим вправо. Доходим до листа, который и даёт искомый индекс. На каждом уровне выбирается ровно один ребёнок, поэтому время $O(\log n)$; двоичный поиск с отдельными запросами сумм занял бы $O(\log^2 n)$.

Для массива $[0, 2, 0, 1, 3, 0, 0, 1]$ и $k = 4$ сумма левой половины равна 3. Переходим вправо с остатком $k = 1$. В диапазоне $[4, 8)$ левая половина $[4, 6)$ имеет сумму 3, поэтому переходим туда. Затем выбираем лист 4 со значением 3. Действительно, сумма до позиции 4 включительно равна 6, а сумма до неё — 3.

Инвариант спуска: искомый элемент находится в текущем диапазоне, а $k$ — порог относительно его начала после вычета пропущенных слева сумм. Неотрицательность обеспечивает монотонность префиксов и позволяет сравнивать порог с полной суммой левой части. С отрицательными числами это неверно: у $[5, -5]$ сумма 0, хотя первый префикс уже достигает 4. Такое левое поддерево нельзя пропустить по одной сумме.

**Практический контракт.** Не вызывайте этот поиск при $k \le 0$, при $k$ больше общей суммы или после обновления, создавшего отрицательный элемент. Можно вместо запрета определить специальный результат «не найдено», но его смысл следует зафиксировать до реализации. Нулевые дополненные листья не создают ложного ответа при допустимом положительном пороге.

**Самостоятельная реализация.** Добавьте поиск $k$-й единицы и точечное переключение 0/1. Сравните ответы с линейным проходом, включая массив из одних нулей, единственную единицу в конце и удаление последней единицы. Объясните, почему основное дерево из этой главы подходит без изменения сводки, а изменяется только алгоритм запроса.

## 2.11. От значений к координатам: подготовка частотного дерева

В предыдущем поиске листья соответствовали позициям исходного массива. Есть другой полезный выбор: лист соответствует возможному значению, а в нём хранится число его вхождений. Тогда сумма префикса — число элементов меньше порога, а поиск по суммам находит порядковую статистику мультимножества. Структура дерева та же, но смысл координаты изменился.

Пусть заранее известны все значения, которые могут появиться: $[100, -7, 100, 40]$. После сортировки и удаления повторов получаем координаты $[-7, 40, 100]$. Значениям сопоставляем ранги 0, 1 и 2. При добавлении 100 увеличиваем частоту листа 2, при удалении уменьшаем её, предварительно проверив, что она положительна. Для мультимножества $\{100, -7, 100, 40\}$ частоты равны $[1, 1, 2]$. Третьему элементу в отсортированном порядке соответствует ранг 2, а исходный ответ — 100.

Сжатие сохраняет порядок и равенство, но не расстояния. Разность рангов значений 100 и 40 равна 1, хотя разность самих значений равна 60. Поэтому в задачах о длине покрытия координатная ячейка должна хранить настоящую длину промежутка, а не единицу. Это различие понадобится в геометрическом приложении следующей главы.

Для запроса «сколько элементов строго меньше 50» сам порог 50 не обязан присутствовать среди координат. `lower_bound` в массиве $[-7, 40, 100]$ возвращает позицию 2; запрашиваем сумму частот на $[0, 2)$ и получаем 2. Для «не больше 100» используем `upper_bound`, получаем позицию 3 и суммируем весь диапазон. Граница принадлежит пространству рангов, а не исходному массиву.

Если все будущие значения известны из входных операций, их можно собрать до построения дерева. Это предварительное чтение не переставляет сами операции: ответы по-прежнему выдаются в их исходном

порядке. Если новые значения приходят в неизвестном заранее потоке, фиксированное сжатие уже не подходит без перестройки; нужны другой интерфейс или другая структура.

**Задание на выбор модели.** Сравните два запроса: «найти третье по величине значение среди всех добавленных» и «найти третье по величине значение только на позициях $[l, r)$ исходного массива». Первому достаточно одного дерева частот. Второй одновременно ограничивает позиции и спрашивает о порядке значений; переход к частотным листьям теряет сведения о позициях. Именно здесь возникает необходимость в более сложных структурах последующих недель.

## 2.12. Практика к главе

В расширенной подборке приложения A: №2090 «Звёздочки» (основная); №4476 «Перестановки» (основная); №2202 «Мегаинверсии» (углубление); №4024 «Очередь в банк» (углубление); №4490 «Восстановление перестановки» (резерв КР). Полные условия доступны по ссылкам в приложении. Выберите основные задачи, затем переходите к углублению; решать весь список за одну неделю не требуется.

## 2.13. Вопросы для самопроверки

1. Зафиксируйте отличие `set(p,x)` от `add(p,x)`.
2. Какие префиксные суммы меняются при изменении позиции 0?
3. Разложите $[1, 7)$ в дереве из восьми листьев на максимальные целиком покрытые узлы.
4. Почему высоты дерева недостаточно для доказательства сложности запроса?
5. Какие узлы меняются после присваивания последнему элементу?
6. Почему построение снизу вверх линейно?
7. Приведите ассоциативную некоммутативную операцию и объясните порядок накопления.
8. Почему среднее следует хранить как пару?
9. Докажите формулу `best` и определите ответ для всех отрицательных значений.
10. Как найти k-й присутствующий элемент по дереву частот? Назовите условие допустимости k и объясните выбор ветви.


(c) Aleksey Tolstikov